Milestone Report

ITI123 Generative AI & Deep Learning Project

Chia Rui Qi, Cherie (8708815V)

1 Motivation

1.1 Problem & Setting

This project addresses the business problem of **predicting the engagement performance of Instagram posts** using historical post data. Engagement is defined as the level of audience interaction a post receives, measured through likes and comments relative to follower count. By predicting engagement levels in advance, businesses can assess their own content or influencer posts, enabling more informed decisions in content planning and marketing campaigns while reducing risk, improving return on investment, and reducing time spent on manual and subjective inspection of historical posts, captions, images and engagement metrics.

1.2 Datasets

Initial dataset

For the initial testing, this project will use a publicly-available Instagram Posts dataset consisting of **1,968 photo posts**, totaling **5,426 images** (Kapadia, 2022). The posts may not necessarily be from influencers. Each post includes: One or more image files (for carousel posts), a caption text file, a compressed JSON metadata file containing post-level information

Dataset Source: Instagram Posts dataset, Kaggle (Kapadia, 2022)

Additional dataset

Another Instagram influencer dataset may be used if it is found that the smaller dataset limits the model's performance (Kim et al. 2020): Instagram Influencer Dataset

This is a larger dataset containing 33,935 influencers with their 10,180,500 Instagram posts (300 posts per influencer). It also contains the following information: caption, usertags, hashtags, timestamp, sponsorship, likes, comments, etc, making it similar to the above dataset.

2 Methods

2.1 Deep Learning Techniques and Architecture

A **multimodal deep learning architecture** is proposed, combining separate networks for different data modalities:

- **Metadata (MLP):** Structured features such as follower count, posting time, and post format flags are processed with fully connected layers and ReLU activations. This captures non-linear interactions and ensures compatibility for later fusion with text and image modules.
- **Text (RNN LSTM and Transformer-based BERT):** Sequential and contextual information in captions are modeled using an LSTM and a pretrained transformer (BERT) for comparison.
  - **LSTM**: Chosen over GRUs (which were mentioned in the Project Proposal) for more expressive gating and finer control over sequential information. Word order and context are preserved through recurrent sequence processing and hidden state propagation across time steps.
  - **BERT**: Evaluated to leverage pretrained bidirectional context and richer semantic representations. Word order is preserved through positional embeddings, while contextual dependencies are modeled via bidirectional self-attention across the entire sequence.
  - Evaluating both models allows comparison of compact, task-specific sequential learning (LSTM) against pretrained, semantically rich contextual learning (BERT) through global self-attention over the entire sequence. This would provide insight into whether task-specific cues or semantic understanding matters more for captions.
- **Image (CNN: ResNet18):** A pretrained ResNet18 extracts visual features, balancing representational power and efficiency. Shallower depth reduces overfitting compared to deeper variants like ResNet50.

The outputs of the all components will be concatenated and passed through dense layers for final engagement classification. This late-fusion design enables each modality to contribute independently and supports clear comparison against unimodal baselines.

2.2 Inputs & Outputs

Inputs: images, text captions, and structured metadata. The table below shows the features extracted or engineered from the Initial Dataset.

Attribute	Extracted/ Engineered	Description	Data type	Usage
user_id	Extracted	ID of the post's owner	int64	For train test split only
following	Extracted	Number of accounts owner is following	int64	MLP
publish_timestamp	Extracted	UTC time of publishing post	datetime 64[ns]	For train test split only
has_location	Extracted	Whether the post has location or not	int64	MLP
is_carousel	Extracted	Whether it post has multiple images or not	int64	MLP
num_images	Extracted	Number of images in the post	int64	MLP
is_sponsored	Extracted	Whether the post is labelled as Paid Partnership or not	int64	MLP
image_path	Extracted	Path to image (or 1st image in carousel)	object	CNN
caption	Extracted	Post caption	object	RNN
follower_following_ratio	Engineered	Number of followers divided by number of accounts following	float64	MLP
hour	Engineered	Hour of publishing post (UTC)	int64	MLP
day	Engineered	Day of publishing post	object	MLP
is_weekend	Engineered	Whether the post was published on a weekend or not	bool	MLP
caption_word_count	Engineered	Number of words in caption, excluding hashtags	int64	MLP
hashtags	Extracted	List of hashtags	object	Unused
num_hashtags	Engineered	Number of hashtags	int64	MLP

While caption-derived statistics (caption_word_count) were included in the metadata encoder, the raw caption text used by LSTM/BERT was processed independently, preventing direct feature leakage between modalities.

Output: engagement_label- a low, moderate or high engagement label on the Instagram post.
Ideally, the engagement label would be split similarly to benchmark ranges of engagement rates found online, where:

- <1% : Low engagement (label 0)
- 1% to 3.5% : Moderate engagement (label 1)
- > 3.5% : High engagement (label 2)

However, when using this split with the smaller dataset with random users, most posts fall within the Low engagement range, and only a few posts have Moderate or High engagement rates.
For early testing with the smaller dataset, the engagement label was instead obtained by splitting the engagement rate into thirds by percentile, keeping the balance in target labels. When using the larger

dataset on influencers, the original intended engagement label would be used, if the target label is not too imbalanced.

2.3 Preprocessing

The number of followers was used to calculate engagement rate, where:

engagement rate = (number of likes + number of comments) / number of followers

Follower-following ratio was also created from the number of followers and account following. After which, the number of likes, number of comments and number of followers were dropped as they directly relate to engagement rate.

From the captions, the number of words and number of hashtags were counted.

Train test split

Data was split temporally within each user, using earlier posts for training and later posts for testing to simulate real-world post evaluation. Data was grouped by user_id and publish_timestamp, and split into 80% for training and 20% for testing. (The split was 70-15-15 for train-validation-test sets in the Project Proposal, but due to the small dataset, I prioritized the training and test sets.) I also filtered out users who had fewer than 3 posts, to avoid overly short histories that would have resulted in 1 post in the train set and test set respectively.

To ensure that the split was done correctly, I checked that the maximum timestamp in the training set was earlier than the minimum timestamp in the test set, for each user. I also checked that the same user appears in both sets. I checked for class imbalance, so that posts in each engagement class have enough samples. I checked that each user has enough posts in each set, by finding the minimum posts of all users in each set. Finally, I plotted engagement rate against time for a random user, to check that the posts are in the correct set.

For MLP

For the Multilayer Perceptron used with the metadata, I one-hot encoded the categorical features (day, hour), and did standard scaling on the numerical features.

For RNN

When using LSTM, I tokenized the captions by splitting on spaces, removing non-alphanumeric characters, and converting all text to lower-case. Then, I built the vocabulary exclusively on the training set to prevent information leakage. Captions in both the training and test sets were then encoded using this fixed vocabulary, with sequences padded or truncated to a maximum length of 64, chosen to be slightly above the 75th percentile of caption word counts after excluding hashtags.

When using BERT, I applied the pretrained BertTokenizer to both the training and test sets. Unlike the LSTM approach, no vocabulary fitting was required, as BERT uses a fixed, pretrained subword vocabulary that is shared across all datasets.

For CNN

For CNN, only the first image of each post was used, as it represents the main visual content influencing engagement and keeps the model efficient. The images were loaded from their relative paths, and transformed images by resizing them to 224px by 224 px, as ResNet was trained on images of 224x224. For data augmentation, random horizontal flip was applied as a light data augmentation technique to improve generalization, and aggressive augmentation such as rotations, cropping or colour changes could remove important visual cues and change the aesthetics of the post. As we are not doing object detection here, there is no need for stronger and unnatural augmentation. Finally, images were normalized using the ImageNet mean and standard deviation to ensure consistency with the data distribution used during ResNet pretraining, which is critical for effective transfer learning.

3 Preliminary Experiments

3.1 Baseline models

The table below summarizes the baseline model architecture:

Modality	Model	Core Architecture	Key Design Choices
Metadata	MLP	40 → 64 → 32 → 3	ReLU activations, Dropout (0.3)

Modality	Model	Core Architecture	Key Design Choices
Text	LSTM	Embedding (50) → LSTM (64) → FC	Sequential modeling, compact embedding, Dropout (0.5), ReLU activations
Text	BERT	Frozen BERT + FC head (768 → 256 → 3)	Pre-trained bidirectional encoder, frozen weights, Dropout (0.5), ReLU activations
Image	ResNet18	Pre-trained ResNet18 + MLP head	Residual CNN, shallow depth, Dropout (0.5), ReLU activations

- MLP: After one-hot encoding, the metadata consisted of 40 numerical and categorical features. The MLP architecture comprises multiple fully connected layers with ReLU activations and dropout regularization, enabling the model to capture non-linear interactions between metadata attributes while remaining computationally lightweight.
- LSTM: The model uses an embedding layer followed by a single-layer LSTM to capture sequential dependencies in captions. The final hidden state is passed through fully connected layers for classification.
  - The embedding dimension was set to 50 to provide a compact semantic representation of words while limiting the number of trainable parameters.
  - The LSTM hidden size was set to 64, which is sufficient to capture sequential patterns in short social media captions while avoiding excessive model complexity.
- BERT: The BERT encoder was frozen, and only task-specific dropout and fully connected layers were trained for classification.
  - The 768-dimensional BERT output is projected to a 256-dimensional bottleneck, acting as a form of regularization and reducing sensitivity to small feature variations compared to directly mapping from 768 to 3.
- ResNet18: The ImageNet-pretrained ResNet18 backbone was frozen, and its original classification layer was replaced with a task-specific multi-layer perceptron head, which was trained from scratch.
  - The 512-dimensional visual feature vector produced by the ResNet18 backbone is projected to a 128-dimensional latent space to balance model complexity and regularization

For LSTM, BERT and ResNet18, ReLU activation was applied to improve non-linear modeling capacity, and dropout was applied to reduce overfitting on limited data.
Both LSTM and BERT were evaluated as text baselines to compare lightweight, task-trained sequential modeling against pretrained contextual language representations. This comparison helps assess whether large-scale pretraining provides meaningful performance gains over simpler recurrent architectures in a low-data setting.

3.2 Evaluation Metrics

In the Project Proposal, Accuracy and macro-averaged F1 scores were selected as the primary evaluation metrics. However, given my three-class classification setting, accuracy may misrepresent model performance, as it hides per-class failure and does not tell how the model is wrong.
Macro-F1 was therefore used as the main metric, to evaluate the harmonic mean of precision and recall for each engagement category (low, moderate, high) and average them equally. This ensures that all classes are evaluated fairly, preventing dominant classes from skewing results and providing a balanced assessment for potentially imbalanced engagement categories.
Confusion matrices were also used to provide additional insight into class-specific strengths and weaknesses.

3.3 Baseline experiment outcomes

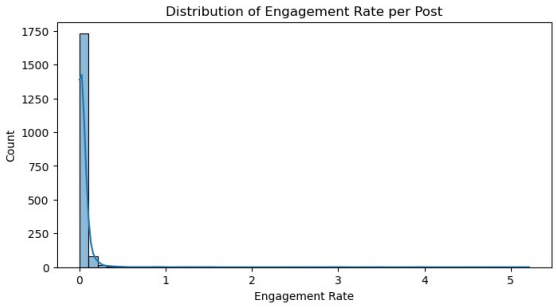
All models were run for 10 epochs, and the accuracy, macro-F1 and confusion matrix of the training set and test set were recorded separately for comparison.

Model	Train/Test	Accuracy	Macro-F1
MLP	Train	60.5%	59.2%
	Test	59.5%	57.7%

Model	Train/Test	Accuracy	Macro-F1
LSTM	Train	51.7%	46.3%
	Test	40.4%	33.2%
BERT	Train	46.3%	43.7%
	Test	49.2%	45.7%
ResNet18	Train	54.0%	53.8%
	Test	52.7%	51.5%

3.4 Error Analysis

Caveat in class distribution



The chart on the right shows the distribution of engagement rates for the posts in the small dataset. Most of the engagement rates are below 1%, and concentrated close to 0%, hence commonly-used engagement benchmarks found online are not applicable. The dataset was instead divided into three classes using an approximately even split by engagement rate quantiles. However, the resulting class ranges remain extremely narrow, with only small numerical differences separating low, moderate, and high engagement posts. :

- <0.006 % : Low engagement (label 0)
- 0.006% to 0.02% : Moderate engagement (label 1)
- > 0.02% : High engagement (label 2)

Such weak class separability could make it difficult to determine which class the post belongs to, resulting in poor performance that is only marginally better than random guessing (33% for 3 classes).

Accuracy VS macro-F1

Across all models, accuracy and macro-F1 are numerically close, but macro-F1 is consistently slightly lower. The small gap between accuracy and macro-F1 indicates that no single class is being completely ignored. Macro-F1 is sensitive to per-class precision and recall and penalizes consistent confusion between classes. The low macro-F1 scores here also suggest that misclassifications frequently occur between adjacent engagement classes, reducing precision and recall for multiple classes simultaneously. This pattern suggests that the model has learned some engagement-related signals but struggles with fine-grained class separation rather than failing to learn useful features altogether.

Overfitting from performance of train VS test set

MLP: Mild overfitting seen from slightly better performance on train set. Metadata provides the best signal among the modalities, but is not enough to fully separate classes.
LSTM: Large performance drop from train to test set, indicating overfitting to captions. This could partially be due to limited linguistic signal in short captions, or from the small dataset, or noisy engagement labels.
BERT: Test performance slightly exceeds train, reflecting natural evaluation variance and indicating a stable but weak learning. The poor results suggests underfitting or limited task-relevant signal. Freezing BERT prevents overfitting, but the pre-trained representations did not transfer well to this engagement task.
ResNet18: Mild overfitting seen from slightly better performance on train set. Visual features provide moderate predictive signal, indicating that images alone are informative but insufficient for precise engagement prediction.

4 Performance tuning and optimization

Model performance was improved through systematic tuning of the parameters below. Manual grid search across these hyperparameters allowed us to select configurations that maximized test macro-F1, while controlling overfitting on the relatively small dataset. I also increased the number of epochs to 50, and implemented early stopping when macro-F1 does not improve after 5 epochs.

Parameter	MLP	LSTM	BERT	ResNet18

hidden_dim	Hd1: 32, **64**, 128 hd2: 16, 32, **64**	32, 64, **128**	-	-
embed_dim		50, **100**		
dropout	**0.3**, 0.5	0.3, **0.5**, 0.6	**0.3**, 0.5	0.3, **0.5**
max_len	-	32, **64**, 128	64, **128**	-
learning_rate	1e-3, **5e-4**	1e-3, **5e-4**	2e-5, **5e-5**	1e-3 (head), **1e-4 (unfreeze backbone)**
freeze	-	-	Frozen, **Partial unfreeze**	Frozen, **Partial unfreeze**

- **Hidden dimension**: Tuned to balance model capacity, avoiding underfitting with small representations and overfitting with overly large ones.
- **Embedding dimension**: Tuned to assess whether representational capacity at the input level limited downstream sequence modeling performance.
- **Dropout rate**: Tuned to control overfitting by reducing co-adaptation of neurons
- **Maximum sequence length**: Tuned to retain sufficient input information while limiting noise, padding, and computational cost.
- **Learning rate**: Tuned to ensure stable and efficient optimization without divergence or slow convergence.
- **Unfreezing strategy**: Tuned to balance leveraging pretrained knowledge and adapting representations to the target task under limited data.
  - BERT: Unfreeze the top 2 encoder layers (final transformer blocks) for fine-tuning, keeping the classification head (fc_hidden + fc_out) trainable, and using layer-specific learning rates (lower for BERT layers, higher for classifier head)
  - ResNet18: Unfreeze the last convolutional block (layer4) for fine-tuning, keeping the fully-connected classifier head trainable, and using separate learning rates (lower for backbone, higher for classifier head)

Model performance is sensitive to random initialization and data sampling, therefore results were reported using a fixed random seed and manual seed for the DataLoader generator for reproducibility. The combinations that resulted in the best macro-F1 on the test set are bolded in the table.

4.1 Experiment outcomes after tuning

MLP: The best macro-F1 score was 59.6%, higher than the baseline performance of 57.7%. There is no overfitting, with training macro-F1 score of 59.5%.

	Train	Test	Observations
Baseline	[[357, 51, 54], [185, 182, 129], [100, 62, 351]]	[[103, 29, 15], [42, 39, 33], [20, 13, 82]]	Moderate engagement is most misclassified. The MLP struggles at separating Moderate engagement from Low and High engagement, suggesting that it has difficulty separating neighbouring engagement levels with small numerical differences.
Tuned	[[354, 56, 52], [179, 181, 136], [95, 59, 359]]	[[104, 29, 14], [39, 44, 31], [19, 14, 82]]	Similar confusion matrices before and after tuning.

LSTM: The best macro-F1 score was 51.6%, higher than the baseline performance of 33.2%. There is severe overfitting, with training macro-F1 score of 81.5%. The top 3 models had similar performance gaps.

	Train	Test	Observations
Baseline	[[219, 2, 241], [43, 59, 394],	[[44, 11, 92], [13, 5, 96],	Heavy misclassification of Low and Moderate engagement as High engagement. The LSTM tends to over-predict higher

	[14, 17, 482]]	[9, 3, 103]]	engagement, likely because textual cues distinguishing engagement are subtle in short captions.
Tuned	[[336, 98, 28], [35, 384, 77], [0, 30, 483]]	[[76, 36, 35], [24, 53, 37], [20, 29, 66]]	Improved classification of Low and Moderate engagement, but worse High engagement classification.

BERT: The best macro-F1 score was 54.4%, much higher than the baseline performance of 45.7%. The top 5 models had similar test macro-F1 scores. The training performance for this best model was not evaluated to save tuning time, limiting direct assessment of overfitting for the best configuration. However, for the 3rd-best model, there is severe overfitting, with training macro-F1 of 71.6% (dropout=0.5, learning_rate=2e-5), while the 5th-best model has no overfitting, with training macro-F1 of 50.3% (dropout=0.5, frozen).

	Train	Test	Observations
Baseline	[[338, 45, 79], [279, 90, 127], [187, 73, 253]]	[[110, 17, 20], [60, 29, 25], [50, 19, 46]]	The model shows strong prediction skew toward the Low engagement class, with both Moderate and High frequently misclassified as Low engagement.
Tuned	Not evaluated to save tuning time	[[107, 23, 17], [44, 43, 27], [42, 12, 61]]	Improved classification of Moderate and High engagement, but prediction skew towards Low still exists.

ResNet18: The best macro-F1 score was 57.6%, higher than baseline performance of 51.5%. All 4 models had similar test macro-F1 scores. There is severe overfitting for the best model, with training macro-F1 score of 98.9%, but other models with similar test macro-F1 had significantly lower training macro-F1 of 75.7% (dropout=0.3) or 59.2% (frozen), with less overfitting.

	Train	Test	Observations
Baseline	[[321, 105, 36], [158, 262, 76], [131, 170, 212]]	[[95, 40, 12], [41, 60, 13], [34, 38, 43]]	Poorer classification of High engagement, distributed across all engagement classes.
Tuned	[[460, 1, 1], [7, 487, 2], [1, 3, 509]]	[[87, 34, 26], [28, 57, 29], [16, 25, 74]]	Improved classification of High engagement, but worse Low and Moderate engagement classification. Equal distribution of misclassifcation of Moderate engagement into Low and High, showing reduced bias to either class.

Conclusions:

- Tuning improves macro-F1 across all models, primarily by better separating Low engagement from the other classes
- Moderate engagement class remains challenging for most models, likely because it lies in a narrow transition region between Low and High engagement, sharing similar features with both classes and lacking distinct, separable signals.
- Most models (LSTM, BERT, ResNet18) benefit from tuning, showing more balanced misclassification of Moderate engagement into Low and High classes and less prediction skew into either class.
- Trade-offs: Improvements in the Low engagement class comes at the expense of recall for the High engagement class, and vice versa.
- Unfreezing BERT and ResNet18 led to slightly higher test macro-F1 scores, but much more overfitting.
- Both LSTM and BERT exhibited overfitting under certain configurations, highlighting the challenges of learning engagement-related signals from short captions and noisy labels.

5 Responsible AI Considerations

5.1 Potential biases/risks

1. **Data distribution bias**
   - The dataset is heavily skewed toward very low engagement posts, with most engagement rates near zero.

- Splitting into three classes via quantiles produces narrow and unevenly distinguishable class boundaries, potentially biasing the models to over-predict certain classes (e.g., LSTM/BERT over-predicting High engagement).
- Risk: Model performance may favor patterns common in the dataset and fail on rare but valid posts with unusually high or low engagement.

2. Representation bias
   - Metadata, captions, hashtags, and images may reflect demographic or content-type biases (e.g., posts from certain geographic regions, content genres, or language styles).
   - Risk: The model might inadvertently favor engagement patterns from overrepresented groups, leading to unfair or misleading predictions.
3. Model bias
   - Unimodal models, particularly LSTM and BERT, show systematic confusion of Moderate engagement, suggesting a prediction skew toward extreme classes.
   - Risk: Users or analysts relying on predicted engagement categories may misinterpret borderline posts, affecting downstream decisions (e.g., marketing prioritization).
4. Interpretability and explainability
   - Deep learning models (LSTM, BERT, ResNet18) are black boxes, making it difficult to understand why a post is classified into a particular engagement category.
   - Risk: Lack of transparency may reduce trust and prevent the detection of unintended biases.

5.2 Mitigation actions

1. Balanced class design
   - Classes were created using quantile-based splitting to reduce class imbalance.
   - Evidence: Confusion matrices show that no class is completely ignored, and macro-F1 scores, while low, indicate that each class contributes to learning.
2. Regularization and validation
   - Dropout layers and lightweight architectures (ResNet18, single-layer LSTM) were used to reduce overfitting, mitigating the amplification of biases from rare training patterns.
   - Evidence: Test performance remains moderately close to training performance for ResNet18 and MLP, showing stable generalization.
3. Modality combination (planned)
   - Using a multimodal fusion model is expected to reduce single-source biases by allowing the model to cross-reference text, images, and metadata, improving fairness and robustness.
   - Evidence: Preliminary unimodal results highlight weaknesses in individual modalities, motivating this mitigation.
4. Transparency measures
   - For future deployment, post-hoc analysis (e.g., feature importance for metadata, attention visualization for text, Grad-CAM for images) can be done to increase interpretability and detect bias patterns.

6 Next Steps

- Explore the larger additional dataset to find Instagram posts with higher engagement rates, to label posts according to common benchmarks and improve class separability
- Choose one model for captions based on better performance, after verifying performance with the additional dataset
- Fusion of the final MLP, selected caption encoder (LSTM or BERT), and CNN models and re-tuning if necessary
- Deploy the trained multimodal metadata + caption encoder + CNN model as a simple web application using Gradio.